E-commerce Database Design and Normalization (UNF to BCNF)

Project Based Learning (PBL) Report
for the course
Database Management Systems – 25CS21003

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING (AIML)

By
G. Angel – 26R15A6603
K. Akshaya – 26R15A6604
P. Shanmuka Sai Prabhath – 26R15A6605
R. Malavika – 26R15A6606
V. Chaithanya – 26R15A6607

Under the guidance of
Dr. K. Arpitha Sreenivasulu



Department of Computer Science and Engineering (AIML)
Accredited by NBA

Geethanjali College of Engineering and Technology
(UGC Autonomous)
(Affiliated to J.N.T.U.H, Approved by AICTE, New Delhi)
Cheeryal (V), Keesara (M), Medchal.Dist.-501 301.

A.Y 2025-26 (Sem-II)

CERTIFICATE

This is to certify that the Project Based Learning (PBL) report entitled “E-commerce Database Design and Normalization (UNF to BCNF)” is a bonafide work carried out by the following students of B.Tech Computer Science and Engineering (AIML), II Year, Geethanjali College of Engineering and Technology (UGC Autonomous), for the course Database Management Systems during the A.Y 2025-26 (Sem-II).

Roll No.	**Name**
26R15A6603	G. Angel
26R15A6604	K. Akshaya
26R15A6605	P. Shanmuka Sai Prabhath
26R15A6606	R. Malavika
26R15A6607	V. Chaithanya

Dr. K. Arpitha Sreenivasulu
Project Guide

Head of the Department
Department of Computer Science and Engineering (AIML)

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. K. Arpitha Sreenivasulu, for assigning this Project Based Learning work and for her guidance and encouragement throughout its completion.

We thank the Head of the Department of Computer Science and Engineering (AIML), the Principal and the management of Geethanjali College of Engineering and Technology for providing the facilities needed to carry out this work, and the faculty members of the department for their support.

Finally, we thank our families and friends for their constant encouragement.

G. Angel
K. Akshaya
P. Shanmuka Sai Prabhath
R. Malavika
V. Chaithanya

TABLE OF CONTENTS

S.No.	Contents	Page No
1	Abstract	ii
2	Introduction	1
3	System Design	4
4	Implementation	13
5	Conclusion	19
6	References	20

ABSTRACT

Database normalization removes redundancy from a database and the update, insertion and deletion anomalies that redundancy causes. This project demonstrates normalization on an e-commerce order database. It starts from one unnormalized relation, Order_UNF, in which the customer, address, shipment and order details are stored together with three repeating groups (order items, payments and returns) held as JSON arrays. The relation is decomposed step by step into First, Second and Third Normal Form, and the result is verified to satisfy Boyce-Codd Normal Form.

Every decomposition is justified with functional dependencies derived from stated business rules, and is checked for lossless join and dependency preservation. The design is implemented in MySQL 8.4 using eight SQL scripts. The final schema has eleven tables with primary keys, foreign keys and CHECK constraints. The tests show that all functional dependencies hold in the data, that the original data can be rebuilt exactly from the final tables by joining them back, and that stored order totals and refunds can be recomputed from the normalized data. The BCNF step needed no further split, and this is reported as it is instead of forcing an artificial decomposition.

1. INTRODUCTION

1.1 About the Project

Database normalization is the process of organizing the tables of a relational database so that each fact is stored in exactly one place. This project demonstrates normalization on an e-commerce order database. It starts from one deliberately unnormalized relation and normalizes it step by step through First Normal Form (1NF), Second Normal Form (2NF), Third Normal Form (3NF) and Boyce-Codd Normal Form (BCNF). Each decomposition is justified with functional dependencies, checked for lossless join and dependency preservation, and then implemented and tested in MySQL.

The aim of the project is to demonstrate normalization correctly, not to build a full commercial e-commerce application. Only the entities needed for this purpose are modelled: customers, addresses, orders, products, sellers, listings, payments, shipments and returns.

1.1.1 Anomalies in the Unnormalized Design

The unnormalized order relation stores the same facts many times, which causes the following problems.

- **Update anomaly.** Arjun’s name, email and phone are stored in both order 1001 and order 1003. Changing the phone number in one row leaves the data inconsistent.
- **Insert anomaly.** A new customer, seller or product cannot be recorded until an order exists for it.
- **Delete anomaly.** Deleting order 1002 also deletes everything known about Priya and her address, because it is her only order.
- **Redundancy.** Product names, seller names, shipper names and city and state values are repeated across rows.
- **Repeating groups.** Items, payments and returns cannot be queried or constrained like ordinary columns.

1.1.2 Business Rules

The functional dependencies used in the project come from the following business rules.

- A customer can place many orders. Each order belongs to one customer and is shipped to one address.
- An order contains one or more items. Each item is a listing, that is, one product sold by one seller at one price. A listing appears at most once per order, and repeated units are recorded in quantity.
- The same product can be sold by several sellers, each through a separate listing.
- An order can have several payment attempts, for example one failed and one successful.
- An order is shipped as one shipment by one shipper (1 order = 1 shipment).
- A return is made against one order item. The refund percentage is entered per return and is not derived from the damage factor.
- A PIN code belongs to exactly one city and state (city means postal district).
- The price of a listing is fixed. This is a simplifying assumption, described in Section 4.2.

1.2 Project Outcomes and Objectives

Objectives

- To start from a deliberately unnormalized e-commerce relation and normalize it through 1NF, 2NF, 3NF and BCNF.
- To identify functional dependencies, candidate keys and primary keys, and to justify every decomposition with them.
- To check each decomposition for lossless join and dependency preservation.
- To implement the final schema in MySQL with primary keys, foreign keys, constraints and sample data.
- To verify through queries and tests that the implementation matches the normalization analysis.

Outcomes

- A schema of eleven tables in 3NF, verified to be in BCNF as well.

- A documented set of functional dependencies, candidate keys and primary keys.
- Lossless-join evidence given both as a mathematical argument and as SQL tests that join the tables back and compare them with the original data.
- Eight reproducible SQL scripts and their test outputs.
- A written list of the assumptions and limitations of the design.

2. SYSTEM DESIGN

2.1 System Architecture

The system is a pipeline of eight MySQL scripts. The source is a single unnormalized table, Order_UNF, in which the three repeating groups (order items, payments and return details) are stored as JSON arrays, because MySQL has no nested relations. Each normalization script reads the tables produced by the previous stage and creates the next stage. The JSON_TABLE function unpacks the repeating groups during the move to 1NF. Fig. 2.1 shows the stages and the number of tables after each one, and Table 2.1 lists the scripts in the order in which they are run.

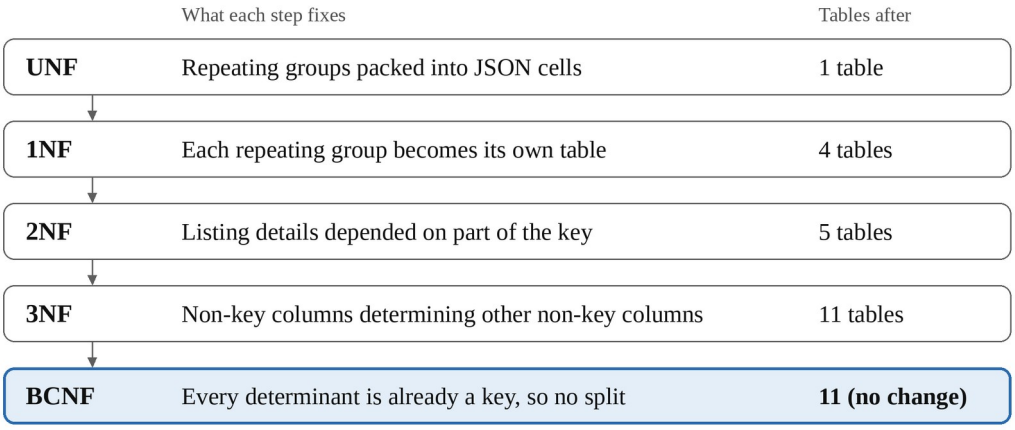


Fig. 2.1 Normalization Stages and Table Counts

All data is moved with plain INSERT statements and never with INSERT IGNORE, so any violation of a key or dependency fails loudly instead of being skipped silently. The first script also drops every downstream table, so the whole pipeline can be run again from scratch. The project uses MySQL 8.4 as the database engine and SQL for schema creation, data insertion, queries and testing.

Table 2.1 Scripts and Their Purpose

Order	File	Purpose
1	unf.sql	Creates the database ecommerce_pbl, resets all tables, creates Order_UNF with repeating groups as JSON arrays and inserts three sample orders
2	ecommerce-check.sql	Validates the source data: item subtotal plus shipping equals the total, and successful payments equal the total
3	ecommerce-1nf.sql	UNF to 1NF
4	ecommerce-fd-analysis.sql	Documents functional dependencies and candidate keys, and tests each dependency against the data
5	ecommerce-2nf.sql	1NF to 2NF, with a lossless-join test
6	ecommerce-3nf.sql	2NF to 3NF; creates and loads the final schema, with lossless-join tests
7	ecommerce-bcnf.sql	BCNF verification of the final schema
8	ecommerce-normalization-report.sql	Row counts, keys, foreign keys, migration checks, a lossless-join PASS/FAIL summary and sample queries

2.2 Modules

Each normal form is treated as a module of the design. The design of every module is described below.

2.2.1 Unnormalized Relation

Order_UNF(order_id, order_date, order_status, customer_id, customer_name, customer_email, customer_phone, address_id, apartment_name, street, city, state, pincode, order_items, payments, shipment_id, shipper_id, shipper_name, tracking_id, shipping_status, shipping_cost, total_order_amount, return_details)

The relation has three repeating groups, each stored as a JSON array in one column.

- **order_items:** listing_id, product_id, product_name, seller_id, seller_name, unit_price, quantity
- **payments:** payment_id, method, status, transaction_reference, amount
- **return_details:** return_id, listing_id, quantity_returned, return_date, damage_factor, refund_percentage, refund_amount

The sample data has three orders. Order 1001 (customer Arjun) has two items and one return, with a total of 7600. Order 1002 (customer Priya) has one item, with a total of 1950. Order 1003 (Arjun again) has two items, including the same product from two different sellers, and a failed payment followed by a successful one, with a total of 5800. In every order the item subtotal plus 100 shipping equals the total, and the successful payments equal the total.

2.2.2 Functional Dependencies and Keys

The dependencies come from the business rules in Section 1.1.2. Data tests can only refute a dependency and never prove it, so the script ecommerce-fd-analysis.sql tests every dependency against the data, and all of them pass. Table 2.2 lists the dependencies and candidate keys.

Table 2.2 Functional Dependencies and Candidate Keys

Relation	Functional dependencies	Candidate keys
Order	order_id → all other attributes	order_id (also shipment_id, since 1 order = 1 shipment)
Order	customer_id → customer_name, customer_email, customer_phone	not a key
Order	address_id → apartment_name, street, pincode, city, state	not a key
Order	pincode → city, state	not a key
Order	shipper_id → shipper_name	not a key
OrderItem	(order_id, listing_id) → quantity	(order_id, listing_id)
OrderItem	listing_id → product_id, product_name, seller_id, seller_name, unit_price	part of a key
OrderItem	product_id → product_name; seller_id → seller_name	not a key
Payment	payment_id → order_id, method, status, transaction_reference, amount	payment_id
Return	return_id → order_id, listing_id, quantity_returned, return_date, damage_factor, refund_percentage, refund_amount	return_id

2.2.3 First Normal Form

Violation

The three repeating groups hold several values in one cell.

Decomposition

Each repeating group becomes its own relation that carries order_id, and all remaining single-valued attributes stay together.

- **Order_1NF**(order_id, order_date, order_status, customer and address attributes, shipment attributes, shipping_cost, total_order_amount) with key order_id
- **OrderItem_1NF**(order_id, listing_id, product_id, product_name, seller_id, seller_name, unit_price, quantity) with key (order_id, listing_id)
- **Payment_1NF**(payment_id, order_id, method, status, transaction_reference, amount) with key payment_id
- **Return_1NF**(return_id, order_id, listing_id, quantity_returned, return_date, damage_factor, refund_percentage, refund_amount) with key return_id

Justification

Every cell is now atomic and every relation has a primary key. The groups are kept separate on purpose. Flattening items, payments and returns into one table would multiply every item by every payment and every return, creating false combinations and redundancy. The data is moved with JSON_TABLE, and the row counts match: 3 orders, 5 items, 4 payments and 1 return.

2.2.4 Second Normal Form

Second normal form applies only where a key has more than one attribute. Order_1NF, Payment_1NF and Return_1NF have single-attribute keys, so partial dependencies are impossible. They are already in 2NF and are left unchanged. OrderItem_1NF has the key (order_id, listing_id), but listing_id → product_id, product_name, seller_id, seller_name, unit_price depends on only part of that key. This is a partial dependency.

Decomposition

- **Listing_2NF**(listing_id, product_id, product_name, seller_id, seller_name, unit_price) with key listing_id
- **OrderItem_2NF**(order_id, listing_id, quantity) with key (order_id, listing_id)

Justification

The decomposition is lossless because the common attribute is listing_id, which is the key of Listing_2NF, so the join can neither create nor lose rows. It

preserves dependencies because (order_id, listing_id) → quantity lies wholly in OrderItem_2NF and listing_id → listing attributes lies wholly in Listing_2NF.

2.2.5 Third Normal Form

Third normal form removes transitive dependencies, where a non-key attribute determines another non-key attribute.

In the order relation (key order_id)

- order_id → customer_id → customer_name, customer_email, customer_phone, so Customer is split out.
- order_id → address_id → apartment_name, street, pincode, and pincode → city, state, so Address and Pincode are split out.
- order_id → shipper_id → shipper_name, so Shipper is split out.
- shipment_id → shipper_id, tracking_id, shipping_status is not a violation. Under the rule 1 order = 1 shipment, shipment_id is itself a candidate key of the order relation (enforced with a UNIQUE constraint), so the shipment data correctly stays in Orders.

In the listing relation (key listing_id)

- listing_id → product_id → product_name, so Product is split out.
- listing_id → seller_id → seller_name, so Seller is split out.

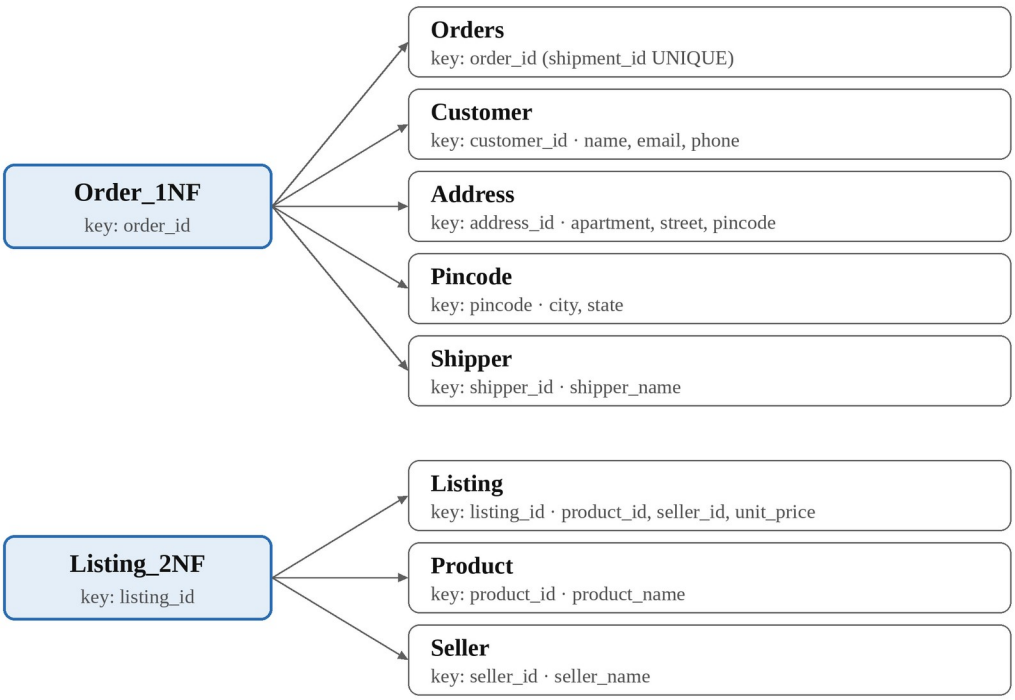


Fig. 2.2 Decomposition into Third Normal Form

OrderItem, Payment and the return relation have no non-key determinants and are already in 3NF. As housekeeping, which is not a normal-form rule, refund_amount is not stored in the final return table because it can be computed as unit_price × quantity_returned × refund_percentage / 100. The report script recomputes it and confirms that it equals the original value.

Every split is made on a determinant that becomes the key of the new relation (customer_id, address_id, pincode, shipper_id, product_id, seller_id), so every join is lossless. Every dependency sits inside one relation, so all dependencies are preserved, and listing_id → product_name still follows by transitivity through Listing and Product.

2.2.6 Boyce-Codd Normal Form

BCNF requires that for every non-trivial dependency X → Y, X is a superkey. Each relation of the 3NF schema was checked, as shown in Table 2.3.

Table 2.3 BCNF Check of the Final Schema

Relation	Determinant	Is it a key
Customer	customer_id	Yes (primary key)
Pincode	pincode	Yes (primary key)
Address	address_id	Yes (primary key)
Shipper	shipper_id	Yes (primary key)
Product	product_id	Yes (primary key)
Seller	seller_id	Yes (primary key)
Listing	listing_id	Yes (primary key)
Orders	order_id; shipment_id	Yes (primary key; UNIQUE)
OrderItem	(order_id, listing_id)	Yes (primary key)
Payment	payment_id	Yes (primary key)
ProductReturn	return_id	Yes (primary key)

The 3NF schema therefore already satisfies BCNF, so no further decomposition is performed and no extra tables are created. A 3NF relation can fail BCNF only when it has overlapping composite candidate keys with a non-key determinant, and none of these relations does. Inventing a violation just to split again would be artificial.

2.2.7 Lossless Join and Dependency Preservation

Table 2.4 summarizes why each decomposition is lossless.

Table 2.4 Lossless Join of Each Decomposition

Decomposition	Common attributes	Why it is lossless
OrderItem_1NF into Listing_2NF and OrderItem_2NF	listing_id	Key of Listing_2NF
Order_1NF into Orders, Customer, Address, Pincode, Shipper	customer_id, address_id, pincode, shipper_id	Each is the key of its new relation
Listing_2NF into Listing, Product, Seller	product_id, seller_id	Each is the key of its new relation

The same property is verified in SQL. The decomposed tables are joined back and compared with the original relation using EXCEPT in both directions, and every comparison returns zero rows, which means that nothing was lost and nothing was invented. The report script also rebuilds the full order header from the UNF data and compares it with the final tables. Dependency preservation holds because every functional dependency in Table 2.2 is contained in a single final relation or follows

from those that are, so all constraints are enforced by primary keys, unique keys and foreign keys without needing joins.

2.3 Backend Design

The backend is a MySQL database named ecommerce_pbl. The final schema, which is both 3NF and BCNF, has eleven tables. Fig. 2.3 shows the entity-relationship diagram, Fig. 2.4 shows the relational schema with all columns and keys, and Table 2.5 lists the primary and foreign keys.

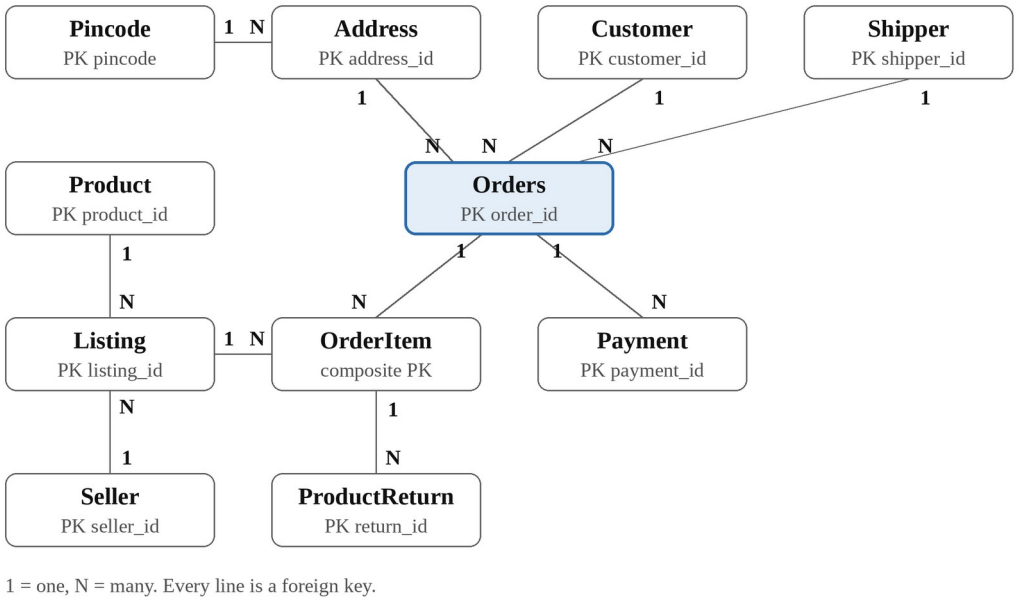


Fig. 2.3 Entity-Relationship Diagram

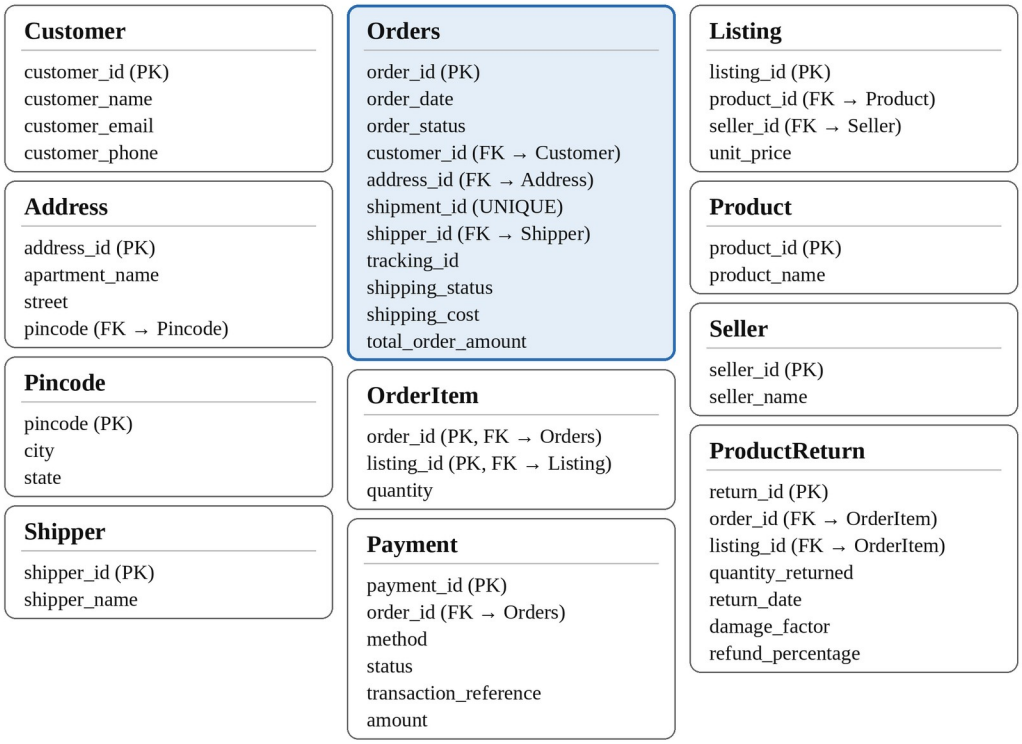


Fig. 2.4 Relational Schema of the Final Database

Table 2.5 Final Schema

Table	Primary key	Foreign keys
Customer	customer_id	none
Pincode	pincode	none
Address	address_id	pincode → Pincode
Shipper	shipper_id	none
Product	product_id	none
Seller	seller_id	none
Listing	listing_id	product_id → Product, seller_id → Seller
Orders	order_id (shipment_id UNIQUE)	customer_id → Customer, address_id → Address, shipper_id → Shipper
OrderItem	(order_id, listing_id)	order_id → Orders, listing_id → Listing
Payment	payment_id	order_id → Orders
ProductReturn	return_id	(order_id, listing_id) → OrderItem

The constraints include NOT NULL on required columns and CHECK (quantity > 0) on item quantities and returned quantities.

3. IMPLEMENTATION

3.1 Modules Implementation

Each module of the design is implemented by one SQL script. The scripts are run in the order given in Table 2.1.

- **unf.sql** creates the database, drops all downstream tables so that the pipeline can be rerun, creates Order_UNF and inserts the three sample orders.
- **ecommerce-check.sql** validates the source data before normalization. For each order it compares the item subtotal plus shipping with the total, and the successful payments with the total, and prints PASS or FAIL.
- **ecommerce-1nf.sql** creates Order_1NF, OrderItem_1NF, Payment_1NF and Return_1NF and fills them from Order_UNF. JSON_TABLE unpacks each repeating group into rows.
- **ecommerce-fd-analysis.sql** lists the functional dependencies and candidate keys and tests each dependency on the 1NF data with COUNT(DISTINCT …) queries.
- **ecommerce-2nf.sql** splits OrderItem_1NF into Listing_2NF and OrderItem_2NF and tests the split with EXCEPT queries in both directions.
- **ecommerce-3nf.sql** creates and loads the final eleven tables with their keys and constraints and tests the splits of the order and listing relations in the same way.
- **ecommerce-bcnf.sql** lists every determinant with its key, and shows the key constraints declared in the database.
- **ecommerce-normalization-report.sql** reports row counts for every stage, the columns, keys and foreign keys, migration checks, a lossless-join PASS/FAIL summary and sample queries that rebuild order totals.

3.2 Sample Code

3.2.1 Unpacking a Repeating Group into 1NF

The order items are moved from the JSON array into OrderItem_1NF with JSON_TABLE.

```
INSERT INTO OrderItem_1NF (
    order_id, listing_id, product_id, product_name,
    seller_id, seller_name, unit_price, quantity
)
SELECT
    o.order_id, i.listing_id, i.product_id, i.product_name,
    i.seller_id, i.seller_name, i.unit_price, i.quantity
FROM Order_UNF AS o
CROSS JOIN JSON_TABLE(
    o.order_items,
    '$[*]' COLUMNS (
        listing_id INT PATH '$.listing_id',
        product_id INT PATH '$.product_id',
        product_name VARCHAR(100) PATH '$.product_name',
        seller_id INT PATH '$.seller_id',
        seller_name VARCHAR(100) PATH '$.seller_name',
        unit_price DECIMAL(10,2) PATH '$.unit_price',
        quantity INT PATH '$.quantity'
    )
) AS i;
```

3.2.2 Splitting the Listing Details into 2NF

The listing attributes that depended on part of the key are moved into their own table.

```
CREATE TABLE Listing_2NF (
    listing_id INT PRIMARY KEY,
    product_id INT NOT NULL,
    product_name VARCHAR(100) NOT NULL,
    seller_id INT NOT NULL,
    seller_name VARCHAR(100) NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL
);

CREATE TABLE OrderItem_2NF (
    order_id INT NOT NULL,
    listing_id INT NOT NULL,
    quantity INT NOT NULL CHECK (quantity > 0),
    PRIMARY KEY (order_id, listing_id),
    FOREIGN KEY (listing_id) REFERENCES Listing_2NF(listing_id)
);
```

```
INSERT INTO Listing_2NF
    (listing_id, product_id, product_name, seller_id, seller_name,
unit_price)
SELECT DISTINCT
    listing_id, product_id, product_name, seller_id, seller_name,
unit_price
FROM OrderItem_1NF;

INSERT INTO OrderItem_2NF (order_id, listing_id, quantity)
SELECT order_id, listing_id, quantity
FROM OrderItem_1NF;
```

3.2.3 Creating the Final Orders Table in 3NF

After the customer, address and shipper attributes are split out, Orders keeps only their keys, which are foreign keys. The UNIQUE constraint on shipment_id records the rule 1 order = 1 shipment.

```
CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    order_date DATE NOT NULL,
    order_status VARCHAR(30) NOT NULL,
    customer_id INT NOT NULL,
    address_id INT NOT NULL,
    shipment_id INT NOT NULL UNIQUE,
    shipper_id INT NOT NULL,
    tracking_id VARCHAR(100) NOT NULL,
    shipping_status VARCHAR(30) NOT NULL,
    shipping_cost DECIMAL(10,2) NOT NULL,
    total_order_amount DECIMAL(12,2) NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
    FOREIGN KEY (address_id)  REFERENCES Address(address_id),
    FOREIGN KEY (shipper_id)  REFERENCES Shipper(shipper_id)
);
```

3.2.4 Lossless-Join Test

The tables are joined back and compared with the original in both directions. A decomposition passes when both queries return zero rows. The second query is the same as the first with its two halves swapped.

```
(SELECT o.order_id, o.listing_id, l.product_id, l.product_name,
        l.seller_id, l.seller_name, l.unit_price, o.quantity
 FROM OrderItem_2NF AS o JOIN Listing_2NF AS l USING (listing_id))
EXCEPT
(SELECT order_id, listing_id, product_id, product_name,
        seller_id, seller_name, unit_price, quantity
 FROM OrderItem_1NF);

-- the same test again with the two SELECT blocks swapped
```

3.3 Sample Output Screenshots and Results

The scripts were run and the outputs are shown below. Fig. 3.1 shows the functional dependency tests, Fig. 3.2 shows the lossless-join and migration checks, and Fig. 3.3 shows the totals rebuilt from the normalized tables together with the row counts at every stage.

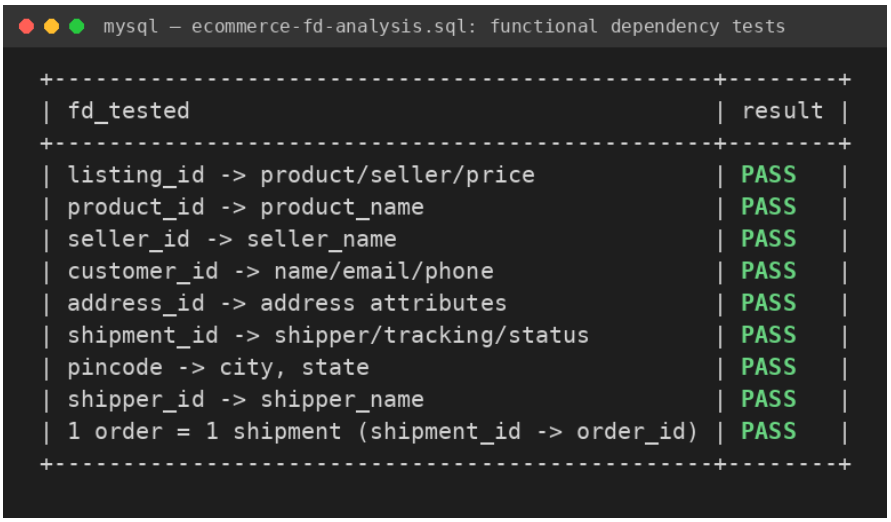


Fig. 3.1 Output of the Functional Dependency Tests

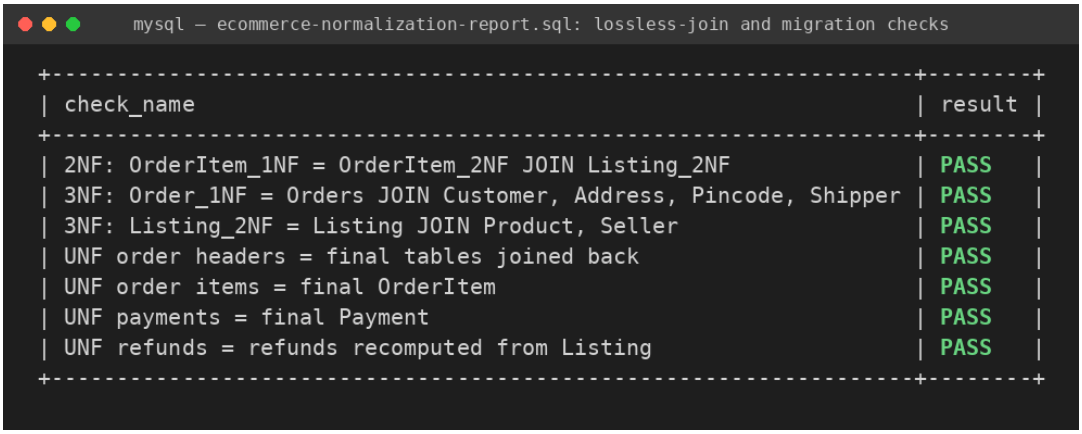


Fig. 3.2 Output of the Lossless-Join and Migration Checks

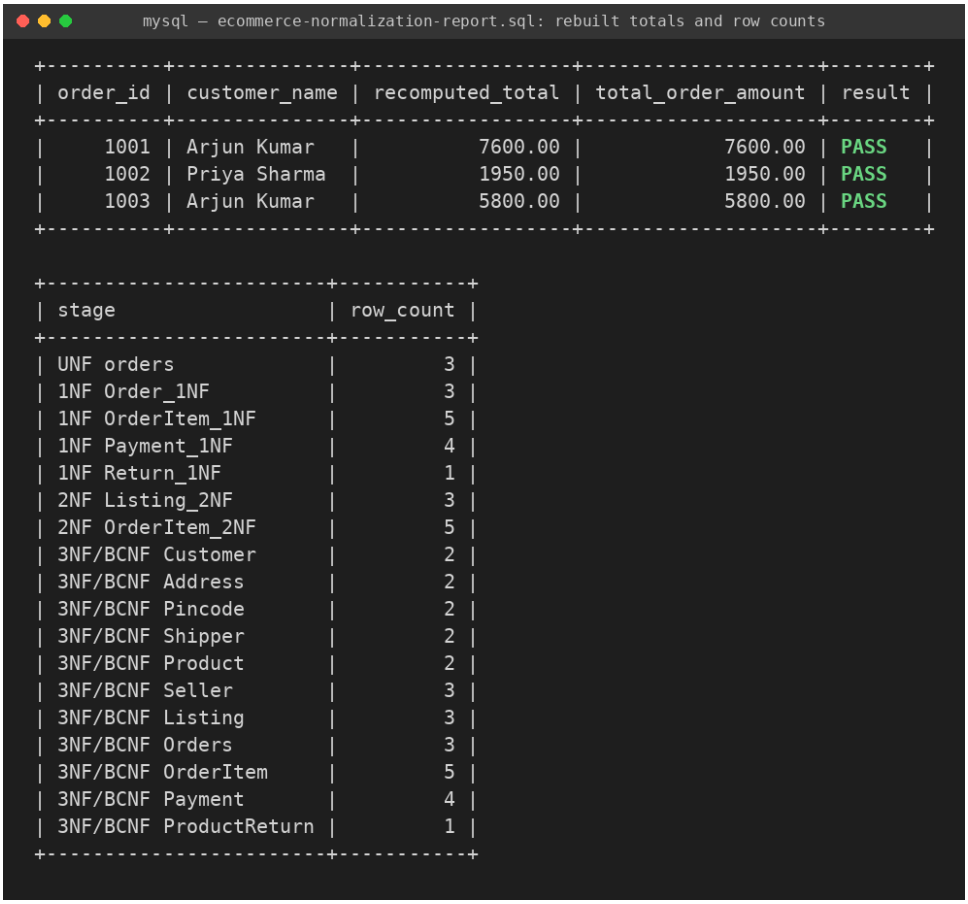


Fig. 3.3 Rebuilt Order Totals and Row Counts at Each Stage

Summary of results

- **Source validation.** For all three orders, the item subtotal plus shipping equals the total, and the successful payments equal the total.
- **Dependency tests.** All nine functional dependency tests on the 1NF data pass.
- **Row counts across stages.** UNF has 3 orders. 1NF has 3 orders, 5 items, 4 payments and 1 return. The final schema has Customer 2, Pincode 2, Address 2, Shipper 2, Product 2, Seller 3, Listing 3, Orders 3, OrderItem 5, Payment 4 and ProductReturn 1.
- **Lossless-join and migration checks.** All seven checks pass, including the rebuild of the order header, the line items and the payments from the original UNF data.
- **Rebuilt totals.** Recomputing each order total from Orders, OrderItem and Listing matches the stored total for orders 1001, 1002 and 1003.
- **Refund check.** The refund recomputed from the final tables equals the original refund_amount of 4000.

- **Negative tests.** Giving one customer a second phone number makes the dependency test report FAIL and the load stop with a duplicate-key error. Changing a stored price or refund makes the lossless-join and refund checks report FAIL. Neither problem is silently hidden.

4. CONCLUSION

4.1 Conclusion

Starting from one relation with three repeating groups, the design was normalized to eleven relations in 3NF. Each step was justified by an actual dependency and never by an invented one. The final schema is already in BCNF, which was shown by checking every determinant instead of forcing another split. The lossless-join tests, the dependency tests and the rebuild queries confirm that the MySQL implementation matches the normalization analysis and that the original data is fully recoverable.

4.2 Limitations and Assumptions

- The dependency listing_id → unit_price assumes that the price of a listing never changes. A real system would store the price paid on each order item.
- 1 order = 1 shipment is a stated simplification. Marketplaces often split multi-seller orders into several shipments.
- pincode → city, state assumes that city means postal district.
- refund_percentage is treated as free input. If it were a fixed function of damage_factor, it would be a further transitive dependency.
- total_order_amount is stored although it can be derived. It is validated by the checks and not removed.
- The sample data is small, so the tests demonstrate the correctness of the design and not its performance.

5. REFERENCES

[1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Pearson, 2016.

[2] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed. McGraw-Hill, 2019.

[3] E. F. Codd, “A Relational Model of Data for Large Shared Data Banks,” Communications of the ACM, vol. 13, no. 6, pp. 377–387, 1970.

[4] Oracle Corporation, MySQL 8.4 Reference Manual. Available at https://dev.mysql.com/doc/refman/8.4/en/